Experiment 5

Create a Library with 4 collections (books, members, borrow Records, authors) and validation rules insert least 10 documents. In this exercise create indexes examples (single-field, compound, text, unique) and show performance test with explain().

k. Create & Use Database and Collections with Validation
l. Insert Sample Documents (10 each)
m. Index Creation
   i. Single-field index
   ii. Compound index
   iii. Text index
   iv. Unique index
n. Performance Test with explain()
   i. Without Index
   ii. With Index
o. Library Database — Additional Indexed Queries
   i. Find books by a specific author quickly
   ii. Search books by title keyword (text index)
   iii. Find overdue books efficiently
   iv. Get borrowing history of a specific member, sorted by date

a. Create & Use Database

```
use LibraryDB


//Books Collection
db.createCollection("books", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["isbn", "title", "authorId", "publishedYear", "copiesAvailable"],
         properties: {
            isbn: { bsonType: "string", description: "must be a string and unique" },
            title: { bsonType: "string" },
            authorId: { bsonType: "int" },
            publishedYear: { bsonType: "int", minimum: 1500, maximum: 2025 },
            copiesAvailable: { bsonType: "int", minimum: 0 }
         }
      }
   }
})


//Members Collection
```

```
db.createCollection("members", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["memberId", "name", "age", "membershipType"],
         properties: {
            memberId: { bsonType: "int" },
            name: { bsonType: "string" },
            age: { bsonType: "int", minimum: 5, maximum: 100 },
            membershipType: { enum: ["Standard", "Premium", "Student"] }
         }
      }
   }
})

//Borrow Records
db.createCollection("borrowRecords", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         required: ["recordId", "memberId", "isbn", "borrowDate", "returnDate"],
         properties: {
            recordId: { bsonType: "int" },
            memberId: { bsonType: "int" },
            isbn: { bsonType: "string" },
            borrowDate: { bsonType: "date" },
            returnDate: { bsonType: "date" }
         }
      }
   }
})

//Authors Collection
db.createCollection("authors", {
   validator: {
```

```
      $jsonSchema: {
        bsonType: "object",
        required: ["authorId", "name", "country"],
        properties: {
          authorId: { bsonType: "int" },
          name: { bsonType: "string" },
          country: { bsonType: "string" }
        }
      }
    }
})
```

b. Insert Sample Documents (10 each)

```
db.books.insertMany([
  { isbn: "B001", title: "MongoDB Basics", authorId: 1, publishedYear: 2020, copiesAvailable: 5 },
  { isbn: "B002", title: "Node.js in Action", authorId: 2, publishedYear: 2019, copiesAvailable: 3 },
  { isbn: "B003", title: "Python Programming", authorId: 3, publishedYear: 2021, copiesAvailable: 7 },
  { isbn: "B004", title: "Java for Beginners", authorId: 4, publishedYear: 2018, copiesAvailable: 4 },
  { isbn: "B005", title: "Data Structures", authorId: 5, publishedYear: 2022, copiesAvailable: 6 },
  { isbn: "B006", title: "Algorithms Unlocked", authorId: 5, publishedYear: 2020, copiesAvailable: 2 },
  { isbn: "B007", title: "Database Systems", authorId: 6, publishedYear: 2017, copiesAvailable: 8 },
  { isbn: "B008", title: "AI and ML", authorId: 7, publishedYear: 2023, copiesAvailable: 10 },
  { isbn: "B009", title: "Deep Learning", authorId: 7, publishedYear: 2021, copiesAvailable: 9 },
  { isbn: "B010", title: "Cloud Computing", authorId: 8, publishedYear: 2019, copiesAvailable: 4 }
])

db.members.insertMany([
  { memberId: 1, name: "Alice", age: 25, membershipType: "Premium" },
  { memberId: 2, name: "Bob", age: 32, membershipType: "Standard" },
  { memberId: 3, name: "Charlie", age: 19, membershipType: "Student" },
  { memberId: 4, name: "David", age: 45, membershipType: "Premium" },
  { memberId: 5, name: "Eve", age: 28, membershipType: "Standard" },
  { memberId: 6, name: "Frank", age: 35, membershipType: "Standard" },
  { memberId: 7, name: "Grace", age: 22, membershipType: "Student" },
  { memberId: 8, name: "Helen", age: 29, membershipType: "Premium" },
  { memberId: 9, name: "Ivy", age: 31, membershipType: "Standard" },
```

```
  { memberId: 10, name: "Jack", age: 27, membershipType: "Premium" }
])

db.borrowRecords.insertMany([
  { recordId: 1, memberId: 1, isbn: "B003", borrowDate: new Date("2025-01-10"), returnDate: new Date("2025-01-20") },
  { recordId: 2, memberId: 3, isbn: "B001", borrowDate: new Date("2025-01-11"), returnDate: new Date("2025-01-21") },
  { recordId: 3, memberId: 2, isbn: "B004", borrowDate: new Date("2025-01-12"), returnDate: new Date("2025-01-22") },
  { recordId: 4, memberId: 5, isbn: "B002", borrowDate: new Date("2025-01-13"), returnDate: new Date("2025-01-23") },
  { recordId: 5, memberId: 4, isbn: "B005", borrowDate: new Date("2025-01-14"), returnDate: new Date("2025-01-24") },
  { recordId: 6, memberId: 6, isbn: "B006", borrowDate: new Date("2025-01-15"), returnDate: new Date("2025-01-25") },
  { recordId: 7, memberId: 8, isbn: "B007", borrowDate: new Date("2025-01-16"), returnDate: new Date("2025-01-26") },
  { recordId: 8, memberId: 7, isbn: "B008", borrowDate: new Date("2025-01-17"), returnDate: new Date("2025-01-27") },
  { recordId: 9, memberId: 9, isbn: "B009", borrowDate: new Date("2025-01-18"), returnDate: new Date("2025-01-28") },
  { recordId: 10, memberId: 10, isbn: "B010", borrowDate: new Date("2025-01-19"), returnDate: new Date("2025-01-29") }
])

db.authors.insertMany([
  { authorId: 1, name: "John Doe", country: "USA" },
  { authorId: 2, name: "Jane Smith", country: "UK" },
  { authorId: 3, name: "Robert Brown", country: "Canada" },
  { authorId: 4, name: "Emily Davis", country: "Australia" },
  { authorId: 5, name: "Michael Wilson", country: "USA" },
  { authorId: 6, name: "Sarah Johnson", country: "India" },
  { authorId: 7, name: "David Lee", country: "China" },
  { authorId: 8, name: "Sophia Martinez", country: "Mexico" },
  { authorId: 9, name: "Chris White", country: "Germany" },
  { authorId: 10, name: "Linda Hall", country: "France" }
])
```

c. Index Creation

i. Single-field index

```
// Single-field index on ISBN
db.books.createIndex({ isbn: 1 })
```

ii. Compound index

```
// Compound index on (memberId, borrowDate)
db.borrowRecords.createIndex({ memberId: 1, borrowDate: -1 })
```

iii. Text index

```
//Text index for book title searches
db.books.createIndex({ title: "text" })
```

iv. Unique index

```
// Unique index on memberId
db.members.createIndex({ memberId: 1 }, { unique: true })
```

v. Performance Test with explain()

d. Without Index

```
db.books.dropIndexes()
db.books.find({ isbn: "B007" }).explain("executionStats")
```

e. With Index

```
db.books.createIndex({ isbn: 1 })
db.books.find({ isbn: "B007" }).explain("executionStats")
```

f. Library Database — Additional Indexed Queries

i. Find books by a specific author quickly

```
// Create index on author for fast lookup
db.books.createIndex({ author: 1 })

// Query to find all books by "George Orwell"
db.books.find({ author: "George Orwell" }).explain("executionStats")
```

ii. Search books by title keyword (text index)

```
// Create text index on title
db.books.createIndex({ title: "text" })

// Search for books containing the word "Mockingbird"
db.books.find({ $text: { $search: "Mockingbird" } }).explain("executionStats")
```

iii. Find overdue books efficiently

```
// Create index on dueDate for borrowings
```

```
db.borrowings.createIndex({ dueDate: 1 })


// Find books that are overdue (before today)
db.borrowings.find({ dueDate: { $lt: new Date() } }).explain("executionStats")
```

iv. Get borrowing history of a specific member, sorted by date

```
// Create compound index for memberId and borrowDate
db.borrowings.createIndex({ memberId: 1, borrowDate: -1 })


// Find member's borrowing history
db.borrowings.find({ memberId: 105 }).sort({ borrowDate: -1 }).explain("executionStats")
```

OUTPUT:

Without Index

"executionTimeMillis" : 12,
"totalDocsExamined" : 500

With Index

"executionTimeMillis" : 1,
"totalDocsExamined" : 1

Overdue Books

{ bookTitle: 'MongoDB Basics', member: 'Anjali', dueDate: ISODate("2025-02-01") }

Borrowing History

{ member: 'Anjali', book: 'DBMS Concepts', borrowDate: ISODate("2025-01-10") }
{ member: 'Anjali', book: 'NoSQL Guide', borrowDate: ISODate("2025-01-25") }

Experiment 6

Demonstrate how sharding improves query performance in MongoDB by comparing query execution on an **unsharded collection** vs a **sharded collection**.

Pre-requisites

- MongoDB sharded cluster is already set up (at least **2 shards**).
- Student is connected to **mongos** router via mongo.

STEP 0 — Start Config Server

Open a terminal and run:

```
mkdir -p /data/configdb
mongod --configsvr --replSet configReplSet --dbpath /data/configdb --port 27019
```

Then open **mongo shell** and initiate replica set:

```
mongosh --port 27019
rs.initiate({
  _id: "configReplSet",
  configsvr: true,
  members: [ { _id: 0, host: "localhost:27019" } ]
})
```

STEP 1 — Start Shard Servers

Start two shard servers:

```
mkdir -p /data/shard1 /data/shard2
mongod --shardsvr --replSet shard1Repl --dbpath /data/shard1 --port 27018
mongod --shardsvr --replSet shard2Repl --dbpath /data/shard2 --port 27020
```

Initiate both replica sets separately:

```
mongosh --port 27018
rs.initiate({ _id: "shard1Repl", members: [ { _id: 0, host: "localhost:27018" } ] })
mongosh --port 27020
rs.initiate({ _id: "shard2Repl", members: [ { _id: 0, host: "localhost:27020" } ] })
```

STEP 2 — Start Mongos Router

```
mongos --configdb configReplSet/localhost:27019 --port 27017
```

STEP 3 — Connect to Mongos

In a new terminal:

```
mongosh --port 27017
```

Add the shards:

```
sh.addShard("shard1Repl/localhost:27018")
sh.addShard("shard2Repl/localhost:27020")
```

STEP 4 — Performance Comparison Script

Run inside mongosh (connected to mongos):

```
//---------------------------------------------------------
// Select Database
//---------------------------------------------------------
use perfTestDB
//---------------------------------------------------------
// Create Unsharded Collection
//---------------------------------------------------------
db.movies_unsharded.drop()

print("Inserting 1,000,000 docs into unsharded collection...")
for (let i = 0; i < 1000000; i++) {
  db.movies_unsharded.insertOne({
```

```
    title: "Movie " + i,
    genre: (i % 5 === 0) ? "Action" : "Drama",
    year: 2000 + (i % 20),
    rating: Math.random() * 10
  })
}
//---------------------------------------------------------
// Create Sharded Collection
//---------------------------------------------------------
sh.enableSharding("perfTestDB")
db.movies_sharded.drop()

print("Creating sharded collection on 'year' field...")
sh.shardCollection("perfTestDB.movies_sharded", { year: 1 })

print("Inserting 1,000,000 docs into sharded collection...")
for (let i = 0; i < 1000000; i++) {
  db.movies_sharded.insertOne({
    title: "Movie " + i,
    genre: (i % 5 === 0) ? "Action" : "Drama",
    year: 2000 + (i % 20),
    rating: Math.random() * 10
  })
}
//---------------------------------------------------------
// Performance Test: Equality Query
//---------------------------------------------------------
print("\n=== Equality Query: year = 2010 ===")
print("Unsharded:")
printjson( db.movies_unsharded.find({ year: 2010 }).explain("executionStats").executionStats
)

print("Sharded:")
printjson( db.movies_sharded.find({ year: 2010 }).explain("executionStats").executionStats
)

//---------------------------------------------------------
// Performance Test: Range Query
//---------------------------------------------------------
print("\n=== Range Query: year between 2010 and 2015 ===")

print("Unsharded:")
printjson(
  db.movies_unsharded.find({ year: { $gte: 2010, $lt: 2015 } })
    .explain("executionStats").executionStats
)

print("Sharded:")
printjson(
  db.movies_sharded.find({ year: { $gte: 2010, $lt: 2015 } })
    .explain("executionStats").executionStats
)

//---------------------------------------------------------
// Aggregation Query
```

```
//-------------------------------------------------------
print("\n=== Aggregation Query: Average Rating by Genre ===")
print("Unsharded:")
printjson(
  db.movies_unsharded.aggregate([
    { $match: { year: { $gte: 2010, $lt: 2015 } } },
    { $group: { _id: "$genre", avgRating: { $avg: "$rating" } } }
  ]).explain("executionStats").stages
)
print("Sharded:")
printjson(
  db.movies_sharded.aggregate([
    { $match: { year: { $gte: 2010, $lt: 2015 } } },
    { $group: { _id: "$genre", avgRating: { $avg: "$rating" } } }
  ]).explain("executionStats").stages
)
```

OUTPUT:

Before Sharding

"executionTimeMillis" : 30,
"totalDocsExamined" : 50000

After Sharding

"executionTimeMillis" : 8,
"totalDocsExamined" : 12000

Experiment 7

Create a simple application that uses MongoDB with python.

Note: To install pymongo package: pip install pymongo

```
import tkinter as tk
from tkinter import messagebox
from pymongo import MongoClient

# MongoDB connection
client = MongoClient("mongodb://localhost:27017/")
db = client["collegeDB"]
collection = db["students"]

# Insert student record into MongoDB
def insert_student():
    name = name_entry.get()
    roll = roll_entry.get()
    branch = branch_entry.get()
    cgpa = cgpa_entry.get()

    if not (name and roll and branch and cgpa):
        messagebox.showwarning("Input Error", "All fields are required!")
        return

    try:
        cgpa = float(cgpa)
        collection.insert_one({"name": name, "roll": roll, "branch": branch, "cgpa": cgpa})
        messagebox.showinfo("Success", "Student record inserted.")
        name_entry.delete(0, tk.END)
        roll_entry.delete(0, tk.END)
        branch_entry.delete(0, tk.END)
        cgpa_entry.delete(0, tk.END)
    except ValueError:
        messagebox.showerror("Invalid Input", "CGPA must be a number.")

# Display all student records
def display_students():
    students = collection.find()
    result_text.delete("1.0", tk.END)
    for s in students:
        result_text.insert(tk.END, f"{s['name']} | {s['roll']} | {s['branch']} | {s['cgpa']}\n")

# GUI setup
app = tk.Tk()
app.title("Student Database App (MongoDB)")

tk.Label(app, text="Name:").grid(row=0, column=0)
name_entry = tk.Entry(app)
name_entry.grid(row=0, column=1)
```

```
tk.Label(app, text="Roll No:").grid(row=1, column=0)
roll_entry = tk.Entry(app)
roll_entry.grid(row=1, column=1)

tk.Label(app, text="Branch:").grid(row=2, column=0)
branch_entry = tk.Entry(app)
branch_entry.grid(row=2, column=1)

tk.Label(app, text="CGPA:").grid(row=3, column=0)
cgpa_entry = tk.Entry(app)
cgpa_entry.grid(row=3, column=1)

tk.Button(app, text="Insert Student", command=insert_student).grid(row=4, column=0, columnspan=2,
pady=10)
tk.Button(app, text="Show All Students", command=display_students).grid(row=5, column=0,
columnspan=2, pady=5)

result_text = tk.Text(app, width=50, height=10)
result_text.grid(row=6, column=0, columnspan=2, padx=10, pady=10)

app.mainloop()
```

OUTPUT:

Python + MongoDB Output

```
{'_id': ObjectId('65f201'), 'name': 'Ravi', 'age': 30}
{'_id': ObjectId('65f202'), 'name': 'Anita', 'age': 25}
```

Open Ended Experiment

i)Description: Identify and implement solution using NO SQL database for scalable unstructured data.

ii) Implementation:

a. **Creation of batches:** students will form group of maximum 3 members as batch to carry out the project.
b. **Synopsis submission:** Students will submit the synopsis for the problem identified that could be solved using machine learning techniques.
addressing societal needs.
c. **Presentation /Demo:** Students will present their project along with report in specified format.